

An efficient chaotic based PSO for earliness/tardiness optimization in a batch processing flow shop scheduling problem

Hadi Mokhtari¹ · Amir Noroozi²

Received: 2 June 2015 / Accepted: 8 October 2015 / Published online: 19 October 2015
© Springer Science+Business Media New York 2015

Abstract The flow shop is a well-known class of manufacturing system for production process planning. The need for scheduling approaches arises from the requirement of most systems to implement more than one process at a moment. Batch processing is usually carried out to load balance and share system resources effectively and gain a desired quality of service level. A flow shop manufacturing problem with batch processors (BP) is discussed in current paper so as to minimize total penalty of earliness and tardiness. To address the problem, two improved discrete particle swarm optimization (PSO) algorithms are designed where most important properties of basic PSO on velocity of particles are enhanced. We also employ the attractive properties of logistic chaotic map within PSO so as to investigate the influence of chaos on search performance of BP flow shop problem. In order to investigate the suggested algorithms, a comprehensive computational study is carried out and performance of algorithms is compared with (1) a commercial optimization solver, (2) a well-known algorithm from PSO's literature and (3) three algorithms from BP's literature. The experimental results demonstrate the superiority of our algorithm against others.

Keywords Manufacturing systems · Chaotic maps · Flow shop · Taguchi technique

Introduction

The need for scheduling approaches arises from the requirement of most systems to implement more than one process at a moment (Wang et al. 2014). When multiple processors are available, then the scheduling will be more complicated, because there is more than one processor which must be kept busy and in effective use at all times. Batch processing is usually carried out to load balance and share system resources effectively and gain a desired quality of service. A *batch processor* (BP) is one in which several jobs can be processed simultaneously by a unique system. We can find real examples of BPs in chemical operations, heat-treating ovens, semiconductor burn-in processes, testing structure of electrical circuits, wafer fabrication, pharmaceutical procedures and construction industries. In such structures, the group of jobs which are processed together is called batch. In many systems, batch structure is usually utilized for cheaper and faster processing of jobs. Batch processing is commonly used for products that are made, very similar and without too much detail. One of the best instances in batch processing manufacturing is car industry. This is because that all cars would have the same body shape and therefore, all can be made at the same time and with no extra cost. There are several advantages of batch production; it can reduce the setup cost because a single production line is used to produce several products. In addition, batch processing can be useful for small businesses who cannot afford to run continuous production lines. One of the main advantages of batch processing is that it can reduce initial capital because a single production line can be used to produce several products. Batch processing has the following characteristics: Commonly used to meet repeated orders; appropriate for product variety; manufacturing equipments are general purpose and suitable for high volume of production. Therefore, the study of scheduling problems

✉ Hadi Mokhtari
mokhtari_ie@yahoo.com; mokhtari_ie@kashanu.ac.ir
Amir Noroozi
amir.noroozie@yahoo.com

¹ Department of Industrial Engineering, Faculty of Engineering, University of Kashan, Kashan, Iran

² Department of Industrial Engineering, Iran University of Science and Technology, Tehran, Iran

with BPs has been attracted considerable attentions. Table 1 shows a brief review on BP scheduling literature. As it can be seen in Table 1, there are some main gaps in BP literature which we express and investigate them in sequel. The most of researches of BPs scheduling addressed single and parallel BPs. Although the importance of flow shop scheduling is obvious (Li and Yin 2013), and many manufacturing environments dealt with a flow shop structure, but very few researches addressed this problem. Damodaran and Srihari (2004) and Manjeshwar et al. (2009) studied a flow shop with two BPs and Lei and Guo (2011) considered this problem with five BPs. The studies in BP flowshop are very few and are limit to two or five BPs. Another drawback is related to the performance measure which is considered for BP problem. From theory viewpoint, the most of the previous works in BP problems (listed in Table 1), considered a regular measure such as makespan, lateness and, etc., while there are important and main non-regular performance measures in theory and practice. From real-world scheduling viewpoint, an important policy of recent manufacturing industries is to use the Just-in-Time systems to increase the flexibility in delivering customer orders as close as possible to the required due dates. However, to the best of our knowledge, only Mönch et al. (2006) investigated such performance measures.

To address the above mentioned drawbacks, we investigate a flowshop with BPs in which the number of BPs is not limited to two or five BPs, and the objective is the minimization of the total earliness and tardiness penalties (henceforth referred to as FBP_E/T). Then, we propose a mixed integer nonlinear formulation of the problem and then obtain a linearized model. Afterwards, to solve the problem, due to the advantages of a particle swarm optimization algorithm (PSO) such as simple structure, immediately accessible for practical applications, ease of implementation, speed to get the solutions, and robustness in the production scheduling problems, we develop improved PSO algorithms.

The remainder of the paper is organized as follows. In “Problem statement” section, after describing the problem in detail, the problem will be formulated. “Solving approach” section develops an improved PSO algorithm and “Experimental design” section describes Taguchi optimization technique and factors of the problem. Data generation and parameter calibration are described in “Data generation and calibration of parameters” section and then “Computational experiments” section compares the algorithms. Finally, “Conclusion and insights” section concludes the paper.

Problem statement

The formal description of a flow shop with m batch processors is as follows. A set of n different jobs are given; each

job j has processing time p_{jm} on BP m . Each processor has a capacity Cap and can process a batch only when the total size of all the jobs does not exceed its capacity. All the jobs have to be grouped into batches and these batches are processed in the same order, on all the processors. Two distinct but dependent decisions should be made: batch formation and batch scheduling. The first is to form batches such that the processor capacity is not violated, and the second is to determine a schedule of batches to optimize the non-regular performance measure. Regarding to the Just-In-Time model considered in this paper, total of earliness and tardiness penalty is the performance measure. We seek the schedule which minimizes $Z = \sum_{j=1}^n \alpha_j \cdot \max \{d_j - C_j, 0\} + \sum_{j=1}^n \beta_j \cdot \max \{C_j - d_j, 0\}$. The completion time of a job (C_j) is the time at which the processing of the batch including that job is finished. The processing time of batch b (B_b) on the processor m which is denoted by P_{bm} is the longest processing times among all the jobs in that batch. To formulate this problem, the problem parameters and decision variables are summarized and listed as follows.

Index list

- j Jobs index ($j = 1, \dots, J$) where J is the number of jobs
- m Processors index ($m = 1, \dots, M$) where M is the number of processors
- b Batches index ($b = 1, \dots, B$) where B is the number of batches

Parameters list

- p_{jm} Processing time of job j on processor m
- S_j Size of job j
- Cap Capacity of processor
- α_j Earliness penalty (/unit/h) of job j
- β_j Tardiness penalty (/unit/h) of job j
- d_j Due date of job j

Decision variables list

- X_{jb} A binary variable indicates the assignment of job j to the batch b

Dependent variables list

- P_{bm} Processing time of batch b on processor m
- C_{bm} Completion time of the b th batch on processor m

Using above parameters and variables, the model of the problem is proposed as a mixed integer nonlinear programming model as follows:

Table 1 A summary of literature

References	Problem	Performance measure	Solution approach*
Wang and Uzsoy (2002)	Single BPM	Maximum lateness	GA
Melouk et al. (2004)	Single BPM	Makespan	SA
Damodaran et al. (2006)	Single BPM	Makespan	GA
Husseinazadeh Kashan et al. (2006)	Single BPM	Makespan	GA
Chou et al. (2006)	Single BPM	Makespan	GA
Mönch et al. (2006)	Single BPM	Earliness/tardiness jobs with a common due date	GA
Damodaran et al. (2007)	Single BPM	Makespan	SA
Husseinazadeh Kashan and Karimi (2007)	Single BPM	Makespan	ACO
Chou (2007)	Single BPM	Makespan	GA
Chou and Wang (2008)	Single BPM	Total weighted tardiness	GA
Mazumdar et al. (2008)	Single BPM	Total completion time	TS
Chang et al. (2004)	Parallel BPMs	Makespan	SA
Koh et al. (2004)	Parallel BPMs	Makespan	GA
Mönch et al. (2005)	Parallel BPMs	Total weighted tardiness	GA
Xu and Bean (2007)	Parallel BPMs	Makespan	GA
Malve and Uzsoy (2007)	Parallel BPMs	Maximum lateness	GA
Husseinazadeh Kashan et al. (2008)	Parallel BPMs	Makespan	GA
Damodaran et al. (2009)	Parallel BPMs	Makespan	GA
Wang and Chou (2010)	Parallel BPMs	Makespan	GA and SA
Damodaran and Srihari (2004)	Flow shop with BPMs (two machines)	Makespan	Mixed integer formulations
Manjeshwar et al. (2009)	Flow shop with BPMs	Makespan	SA
Lei and Guo (2011)	Flow shop with BPMs (up five machines)	Total tardiness, maximum, tardiness and number of tardy jobs	VNS

*GA genetic algorithm, SA simulated annealing, TS Tabu search, VNS variable neighborhood search, ACO ant colony optimization

$$\begin{aligned}
 \min \sum_{j=1}^J \left[\alpha_j S_j \max \left(0, d_j - \sum_{b=1}^B (X_{jb} C_{bm}) \right) \right] & \quad m = \{2, 3, \dots, M\}, \quad b = \{2, 3, \dots, B\} \quad (7) \\
 + \sum_{j=1}^J \left[\beta_j S_j \max \left(0, \sum_{b=1}^B (X_{jb} C_{bm}) - d_j \right) \right] & \quad X_{jb} \in \{0, 1\}, \forall j = \{1, 2, \dots, J\}, b = \{1, 2, \dots, B\} \quad (8) \\
 & \quad C_{bm} \geq 0, P_{bm} \geq 0, \quad \forall b = \{1, 2, \dots, B\}, m = \{2, 3, \dots, M\}. \quad (9)
 \end{aligned}
 \tag{1}$$

Subject to

$$\sum_{b=1}^B X_{jb} = 1, \forall j = \{1, 2, \dots, J\} \tag{2}$$

$$\sum_{j=1}^J S_j X_{jb} \leq Cap, \quad \forall b = \{1, 2, \dots, B\} \tag{3}$$

$$\begin{aligned}
 P_{bm} &= \max_j (p_{jm} X_{jb}), \forall b = \{1, 2, \dots, B\}, \\
 m &= \{1, 2, \dots, M\} \tag{4}
 \end{aligned}$$

$$C_{0,m} = 0, \forall m = \{1, 2, \dots, M\} \tag{5}$$

$$C_{k,0} = 0, \forall k = \{2, 3, \dots, K\} \tag{6}$$

$$C_{bm} = P_{bm} + \max (C_{b-1,m}, C_{b,m-1}),$$

The earliness and tardiness penalties are minimized by the objective function (1) in which C_{bm} is the completion time of b th batch in sequence on the last processor. Each job is assigned to just one batch by constraint (2). Constraint (3) ensures that the sum of sizes of jobs in a batch is less than processor capacity. Constraint (4) enforces the batch processing time to be equal to the longest processing time of all the jobs in the batch. Constraints (5) and (6) compute the completion time of the batches in sequence. Constraint (7) computes completion time of the b th batch in sequence on the processor m . Constraint (8) enforce binary restriction on X_{jb} decision variable and Finally constraint (9) enforce the non-negativity requirement on C_{bm} , and P_{bm} , dependent variables.

The problem formulated above is a mixed integer non-linear programming (MINLP) model which is entitled P_1 hereafter. In order to solve such a problem efficiently, it should first be linearized. To this end, we first replace objective function (1) with a new one and a new constraint as follows.

$$\text{Min} \sum_{j=1}^J \alpha_j S_j E_j + \sum_{j=1}^J \beta_j S_j T_j \quad (10)$$

$$T_j - E_j = \sum_{b=1}^B (X_{jb} C_{bM}) - d_j \quad \forall j = \{1, 2, \dots, J\} \quad (11)$$

where E_j and T_j are new decision variables which indicate earliness and tardiness of job j , respectively.

However, the constraint (11) has still a nonlinear term $X_{jb} C_{bM}$ in which $X_{jb} \in \{0, 1\}$ and $C_{bM} \geq 0$. To linearize this term, let us introduce new variable $z_{jb} = X_{jb} C_{bM}$ for $j = \{1, 2, \dots, J\}$ and $b = \{1, 2, \dots, B\}$ and additional constraints:

$$z_{jb} \leq M_1 X_{jb} \quad \forall j = \{1, 2, \dots, J\}, b = \{1, 2, \dots, B\} \quad (12)$$

$$z_{jb} \leq C_{bM} \quad \forall j = \{1, 2, \dots, J\}, b = \{1, 2, \dots, B\} \quad (13)$$

$$z_{jb} \geq C_{bM} - (1 - X_{jb}) M_1 \quad \forall j = \{1, 2, \dots, J\}, b = \{1, 2, \dots, B\} \quad (14)$$

$$z_{jb} \geq 0 \quad \forall j = \{1, 2, \dots, J\}, b = \{1, 2, \dots, B\} \quad (15)$$

where M_1 is a big positive number. Note that if X_{jb} is zero, then the constraint (12) ensures that z_{jb} will be zero. In such situation, the constraint (14) only states that z_{jb} has to be greater than a negative number. On the other hand, if X_{jb} is 1, the constraint (12) ensures that z_{jb} is less than big M , which is further tightened by constraint (13). The constraint (14) states then that z_{jb} has to be greater than or equal to C_{bM} , exactly as we wanted ($z_{jb} = C_{bM}$). Hence we can replace constraint (11) with constraints (12)–(15) and:

$$T_j - E_j = \sum_{b=1}^B (z_{jb}) - d_j \quad \forall j = \{1, 2, \dots, J\} \quad (16)$$

However, the resulting model is not a MILP. The constraints (4) and (7) have still a nonlinear term, i.e., $\max(\cdot)$ function. In general, if we have a function $\max\{x_1, x_2, \dots, x_n\}$ for continuous variables $x_1, x_2, \dots, x_n$, we can replace $\max\{x_1, x_2, \dots, x_n\}$ with a new variable Y by adding constraints:

$$Y \geq x_i \quad \forall i = 1, 2, \dots, n \quad (17)$$

$$Y \leq x_1 \text{ or } Y \leq x_2 \text{ or } \dots \text{ or } Y \leq x_n \quad (18)$$

or equivalently

$$Y \geq x_i \quad \forall i = 1, 2, \dots, n \quad (19)$$

$$Y \leq x_1 + M(1 - y_1) \quad (20)$$

$$Y \leq x_2 + M(1 - y_2) \quad (21)$$

$\vdots$

$$Y \leq x_n + M(1 - y_n) \quad (22)$$

$$y_1 + y_2 + \dots + y_n = 1 \quad (23)$$

$$y_i \in \{0, 1\} \quad \forall i = 1, 2, \dots, n \quad (24)$$

where M is a big positive number and $Y \geq 0$. By using similar way, can replace constraint (4) with:

$$P_{bm} \geq p_{jm} X_{jb} \quad \forall b = \{1, 2, \dots, B\}, m = \{1, 2, \dots, M\} \quad (25)$$

$$P_{bm} \leq p_{jm} X_{jb} + M_2(1 - u_{bm}) \quad \forall b = \{1, 2, \dots, B\}, m = \{1, 2, \dots, M\} \quad (26)$$

$$\sum_{b=1}^B \sum_{m=1}^M u_{bm} = 1 \quad (27)$$

$$u_{bm} \in \{0, 1\} \quad \forall b = \{1, 2, \dots, B\}, m = \{1, 2, \dots, M\} \quad (28)$$

and constraint (7) with:

$$C_{bm} = P_{bm} + W_{bm} \quad \forall m = \{2, 3, \dots, M\}, b = \{2, 3, \dots, B\} \quad (29)$$

$$W_{bm} \geq C_{b-1,m} \quad \forall m = \{2, 3, \dots, M\}, b = \{2, 3, \dots, B\} \quad (30)$$

$$W_{bm} \geq C_{b,m-1} \quad \forall m = \{2, 3, \dots, M\}, b = \{2, 3, \dots, B\} \quad (31)$$

$$W_{bm} \leq C_{b,m-1} + M_3 v \quad \forall m = \{2, 3, \dots, M\}, b = \{2, 3, \dots, B\} \quad (32)$$

$$W_{bm} \leq C_{b,m-1} + M_3(1 - v) \quad \forall m = \{2, 3, \dots, M\}, b = \{2, 3, \dots, B\} \quad (33)$$

$$W_{bm} \geq 0 \quad \forall m = \{2, 3, \dots, M\}, b = \{2, 3, \dots, B\} \quad (34)$$

$$v \in \{0, 1\} \quad (35)$$

Now we obtain a mixed-integer linear programming (MILP) version of problem, entitled P_2 .

Solving approach

The problem we are addressing in current paper is NP-hard. It follows from NP-hardness of the single machine problem of minimizing the earliness/tardiness (Lauff and Werner 2004). Hence it cannot be solved exactly in polynomial time, and hence we are going to design an efficient PSO for solving

it. PSO is a population-based optimization algorithm which introduced by Kennedy and Eberhart (1997) to optimize various functions. Due to its easy implementation and excellent performance, PSO has been gradually applied to many optimization problems during recent years. One of the advantages of PSO is that PSO is easy to implement. In addition, PSO takes real numbers as particles. It does not require derivatives and uses smaller time steps. Moreover, PSO is easily parallelizable and simpler to understand and implement. It has few parameters to calibrate and needs lower computational costs. Unlike genetic algorithm, PSO has flexibility to control the balance between global and local search which enables it to overcome the premature convergence and enhance the search capability. In PSO, each member is called *particle* and each population of particles is called *swarm*. Each particle is randomly or heuristically initialized. At each step of the optimization, particles combine information from their previous best position (*Pbest*) and their neighborhood best position (*Gbest*) to maximize the probability that they are moving toward a region of space. PSO has two primary operators: *Velocity* and *Position*. At each iteration, a new velocity is computed based on current velocity of each particle, the distance from its *Pbest*, and the distance from *Gbest*. The new velocity value is then used to compute the next position of the particle in the search space according to the following equations:

$$v_{ij}^{t+1} = wv_{ij}^t + C_1r_{1j}^t(Gbest - x_{ij}^t) + C_2r_{2j}^t(Pbest - x_{ij}^t) \quad (36)$$

$$x_{ij}^{t+1} = x_{ij}^t + v_{ij}^{t+1} \quad (37)$$

for $i = 1, \dots, PopSize$ and $j = 1, \dots, n$, where *PopSize* is the total number of particles and n is dimension of the search space. x_{ij}^t and v_{ij}^t are the position and velocity of dimension j of particle i at iteration t , respectively. Parameters C_1 and C_2 are *acceleration coefficients*, and r_{1j}^t and r_{2j}^t are parameters on interval (0, 1). The w is *inertia weight* which controls the impact of the previous velocities on the current velocity.

PSO algorithm for FBP_E/T

As mentioned before, in PSO algorithm, each particle moves towards its *Pbest* and *Gbest*. In suggested PSO for the FBP_E/T, the parameters were initialized and the initial population was generated randomly using Eqs. (19) and (20) as follows.

$$x_{ij}^0 = x_{min} + (x_{max} - x_{min}) \times r_1 \quad (38)$$

$$v_{ij}^0 = v_{min} + (v_{max} - v_{min}) \times r_2 \quad (39)$$

where x_{min} and x_{max} are lower and upper bounds of particle positions, v_{min} and v_{max} are lower and upper bound of particle velocity, and r_1 and r_2 are uniformly distributed random parameters between (0, 1). Since the random parameters (r_1, r_2) have great importance and influence on performance of first solutions and hence final ones, a chaotic approach will be employed in next section to determine these parameters effectively. The velocity values are restricted to range $[v_{min}, v_{max}]$ in order to reduce the likelihood the particle might leave the search space. Here $x_{min} = 0$ and $x_{max} = 1$ and $[v_{min}, v_{max}]$ are determined in calibration section using Taguchi optimization technique.

Evaluation of each particle requires the determination of the permutation of jobs for the FBP_E/T. Since Tasgetiren et al. (2007) reported good performance for the smallest position value (SPV) rule; we adapt this rule to convert the continuous solutions to the discrete ones. Moreover, in this section, we apply the SPV rule in first-first (FF) heuristic (Damodaran et al. 2006) for batch formation as follows. Assign job 1 to the lowest RK. Similarly, for jobs 2 to n assign them to the lowest available RK. Afterwards, for batch formation, place the first job in the list in batch 1. Suppose we are placing the job j , job j is placed in the lowest indexed batch if the size of job does not exceed the remaining capacity of the batch (i.e., $s_j \leq Cap - \sum_{k \in J, k \neq j} s_j X_{kb}$). This process repeats to allocate all jobs to batches. Besides, the sequence of batches is based on the batch number, that is *Batch 1, Batch 2, ..., Batch B*.

Chaotic parameters

The chaos as a kind of dynamic behavior of nonlinear systems has attracted many interests in different fields of sciences. A nonlinear system is said to be chaotic if it exhibits sensitive dependence on initial conditions, and includes infinite number of unstable periodic motions (Yuan et al. 2002). As a relatively new application of chaos, the chaotic maps can be incorporated with random optimization methods to improve the global convergence by escaping from local results. It can be carried out by using such a chaotic map instead of random parameters used in optimization algorithms. It is capable of carrying out an enriched search than stochastic methods with probabilistic nature. Since chaos shows non-repetition behavior, it can perform overall searches at higher speeds as opposed to the traditional random searches. Hence, we are going to design a chaotic search to further enhance the quality of solutions attained by the main PSO algorithm. This algorithm uses chaotic numbers instead of random ones. In literature, there are different ways of embedding chaos into PSO. In sequel, we review some of main approaches presented so far. Xiang et al. (2007) suggested an approach where problem variables are directly selected by Chaos during all iterations of a classic PSO and employed

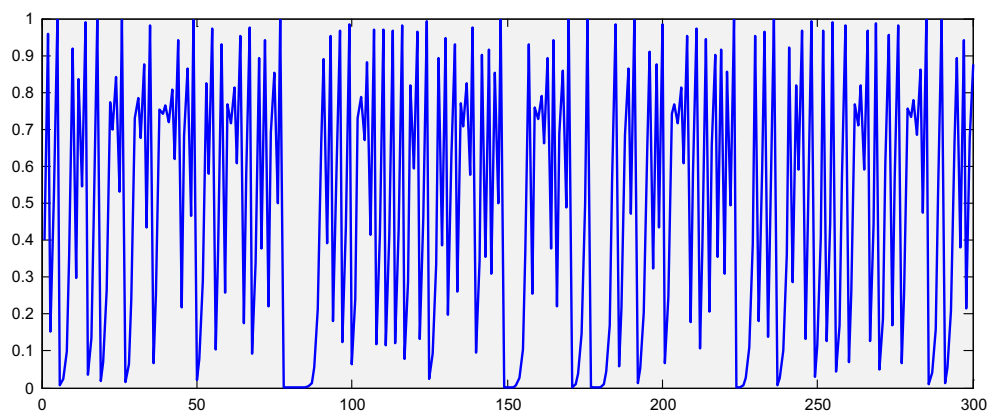


Fig. 1 The logistic chaos numbers with $X_0 = 0.4$

a piecewise linear map in their approach. In another work, Santos Coelho (2008) considered a quantum-based mutation operator for PSO and improved the mutation via Zaslavskii map. But, Alatas et al. (2009) is a more perfect approach. In their paper, 12 methods for embedding Chaos into classic PSO (CEPSO1~CEPSO12) are proposed and investigated where value of parameters r_1 , r_2 , c_1 , c_2 and w are selected via chaotic maps. The initial velocity and positions are also proposed to be directly generated via chaos. Moreover, Wang et al. (2010) proposed an enhanced PSO algorithm to solve dynamic economic dispatch problem. To avoid the premature convergence, chaotic mutation is designed by chaos. Cheng et al. (2012) studied a special chaos embedded PSO in which chaos set G_{best} of PSO as initial point of chaotic sequences and then the chaotic sequences convert to a number of particles around G_{best} . Tang et al. (2013) studied a Wireless video sensor network and devised a chaotic PSO for resource control and lifetime optimization, where chaos is utilized to initialize the population and diversify the search process. He et al. (2014) designed a chaotic search by logistic map within classic PSO for solving a reservoir flood control. In another work, Turgut et al. (2014) proposed a new position update procedure for a quantum behaved PSO algorithm and updated its parameters via chaos variables. A voltage stability boundary problem is considered by Kuru et al. (2015) in electrical power systems where chaotic sequences have been adopted for PSO instead of random sequences to control the values of the parameters in velocity update during whole iterations. Since one of the frequently used chaotic maps in literature is logistic (Mark Berliner 1992; Gutierrez and Iglesias 1998; Dauxois et al. 2002; Iqbal et al. 2014; Mokhtari and Salmasnia 2015), we also employ it in our approach. The logistic map is a polynomial mapping, often described as an example of how chaotic behavior can arise from very simple non linear equations. It is one of the simplest maps which was first introduced by May (1976) for describing nonlinear dynamics of biological population. Similar to other chaotic maps,

this map is a recursive relation for getting a number from previous number. The mathematical form of logistic map is as follows.

$$X_{i+1} = aX_i(1 - X_i) \quad (40)$$

in which X_i indicates i th chaotic number and i represents the iteration number. By setting $a = 4$, $X_0 \in (0, 1)$ and $X_0 \neq \{0, 0.25, 0.5, 0.75, 1\}$ we have $X_i \in (0, 1)$. We use logistic chaotic numbers instead of random numbers for producing r_1 and r_2 so as to benefit the quasi-stochastic behavior that differs from the common random mechanism and also gain non-repetition behavior which can perform overall searches at higher speeds as opposed to the traditional random searches. For example, Figs. 1 and 2 show chaos numbers generated with $X_0 = 0.4$ and $X_0 = 0.6$ respectively.

Improved discrete PSOs

In this section, we investigate two most important and dangerous properties of classical PSO algorithm and employ some suggestions and modifications to enrich both properties.

IDPSO_SC

A property of PSO, which leads to PSO plunge into a local optimum is that, before particles reach to around of the best solution, their velocities come zero and particles are stopped (Van den Bergh 2006). So, if particles can reach around the best solution at a minimum time, before their velocity reach to zero, the probability of finding the better solution will be increased. Consider the following example.

A number of people search for a treasure. Each of them uses a charging treasure detector and their searches are based on the PSO algorithm principals to explore and move in the space regularly. After a while, some of them understand that

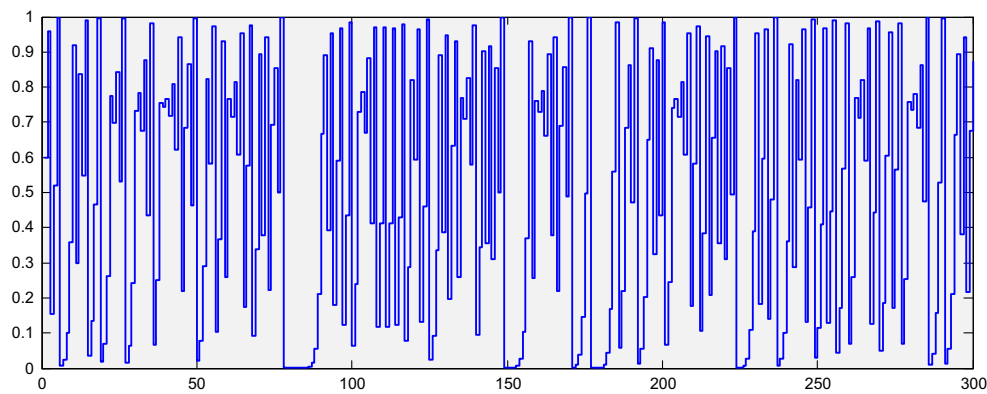


Fig. 2 The logistic chaos numbers with $X_0 = 0.6$

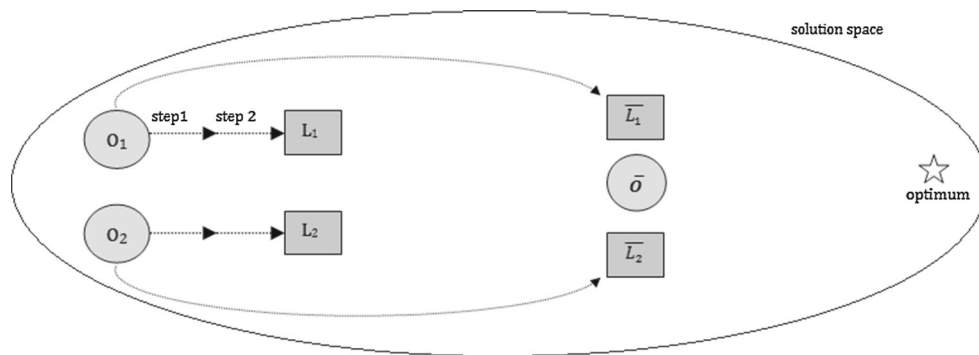


Fig. 3 Comparison between persons' movement using PSO and improved PSO

have bad movement compared with other persons and know that if they reach near persons who have good movement, they may do a better search. But, because they should take steps according to PSO algorithm principals, they have to take step and search slowly towards persons who have had good movement. In these conditions, if these persons are away from the treasure point, the charge of their treasure detector will be finished (terminate) before reaching to around the treasure point or persons who have had good movement. Therefore, if these persons can reach themselves to people who have had good movement, the charge of their treasure detector not wasted and can help other people to explore the treasure point. This example is depicted by Fig. 3. In this figure, persons O_1 and O_2 are far from the person $\bar{O}$ (the person $\bar{O}$ is closer to optimum point). Treasure detector distinguishes which person is close to treasure point). In this state, using PSO principals and after two steps, persons O_1 and O_2 can reach to locations L_1 and L_2 , only.

In the above example, persons are particles of PSO, the charge of the treasure detector is the velocity of particle, position of people in search space is the position of particle, the treasure point is optimum point and treasure detector is the objective value of a problem which distinguishes a particle is close to optimum point or far. In order to modify the above stated conditions in PSO algorithm and to pre-

vent the wasting of velocity (charge of treasure detector), we embed selection mechanism and crossover operator of GA algorithm in IDPSO_SC. Using the selection mechanism, the algorithm finds the “ $1-Reproduct_cross$ ”% of the best solutions through the population and uses crossover operation to reproduce “ $Reproduct_cross$ ”% of the population as new particles (positions of these particles are produced closer the better solutions). Then, the worst solutions replaced with the new particles. Using selection mechanism, people which had good movement are found, then with crossover operator, positions are produced around these people, and finally positions of people which had bad movement are replaced with these new produced positions. This process is demonstrated in Fig. 3. In this figure, using the new suggestions and modifications, persons O_1 and O_2 can reach to locations $\bar{L}_1$ and $\bar{L}_2$ (new produced positions closer to better points) and help to the person $\bar{O}$ to find the optimum point.

At the beginning of the PSO algorithm; particles move fast and find some feasible solutions. Since after several iterative generations, particles need to search the space near the current $Pbest$ and $Gbest$ carefully, both population diversity and convergence capacity should be kept during the iterative process. This approach is implemented using a function of w . In IDPSO_SC, w is considered to be declined at the end of each iteration as follows.

```

Initialization of positions and velocities of swarm;
Iteration = 2; //  $\log_{10}(1)$  is equal to 0
Repeat
    Evaluate fitness of each particle;
    Update  $Pbest$ ;
    Update  $Gbest$ ;
    Sorting solutions in increasing order of objective function values; // The more desirable solution, the smaller
                                                                    objective function value.

    Select  $Reproduce\_Cross\%$  of solutions of population through “1-  $Reproduct\_cross$ ”% of best ones;
    Apply crossover on selected solutions;
    For  $i=1- Reproduct\_cross$  to  $Popsiz$ e
        Replace  $X_i^{iteration}$  with new produced positions by crossover

     $w = \frac{\omega}{\log_{10}^{iteration}}$ ;
    iteration = iteration + 1;
    Update velocities; //  $V_{ij}^{t+1} = wV_{ij}^t + C_1r_{1j}^t(Gbest - X_{ij}^t) + C_2r_{2j}^t(Pbest - X_{ij}^t)$ ;
    Update positions; //  $X_{ij}^{t+1} = X_{ij}^t + V_{ij}^{t+1}$ ;

Until stopping criterion is not met;

```

Fig. 4 Pseudo code of IDPSO_SC

$$w = \omega / \log_{10}(\text{iteration}) \quad (41)$$

where ω is a constant coefficient which is assigned by user. Pseudo code of IDPSO_MC indicated in Fig. 4.

Since the effective performance of PSO algorithm depends on the adaptive parameters, the appropriate C_1 , C_2 and ω are calibrated with taguchi optimization technique (described in subsequent sections).

IDPSO_MC

Van den Bergh (2006) stated a dangerous property of the PSO algorithm, which all the different versions of the PSO algorithm have this dangerous property. if $x_{ij}^t = Pbest = Gbest$, then the velocity update will depend only on wv_{ij}^t . In other words, if the particle's current position coincides with the $Gbest$ position or $Pbest$ position, the particle will only move away from this point if its pervious velocity and w are non-zero. If their pervious velocities are very near to zero, then all the particles will stop moving once they catch up to the $Gbest$, which may lead to premature convergence of the algorithm. In fact, all the particles have converged on the $Pbest$ discovered so far by the swarm. To obviate this drawback, at each iteration of IDPSO_MC, if $Pbest$ changed and updated, a mutation operator is applied on x_{ij}^t corresponding to this $Pbest$. With this operation, the velocity will not depend only on wv_{ij}^t , because of the equation $Pbest - x_{ij}^t$ is non-zero. This operation should not be applied on $Gbest$, because of, lead to divergence and perverting particles. Another important operation is application of the crossover operator on velocity. This

operation will cause the diversity of particles' velocity in different dimensions. Suppose that the movement of a particle in a dimension direction is good, but due to the low velocity, particles move slowly in that direction. Whereas, if the velocity is the higher, then particle will move the better. By applying the crossover on velocity, particle can move with a good velocity in that direction. Furthermore, to reach more diversity at each iteration, we apply the mutation operator on $Reproduct_rate\%$ of population randomly. We also set Eq. (21) to decrease w synchronous at each iteration. Pseudo code of IDPSO_MC indicated in Fig. 5.

Experimental design

The different levels of the parameters strongly affect the quality of the solutions obtained by a PSO algorithm. Most users adjust parameters manually. Such trial-and-error method is usually time consuming, ineffective, and often it could not locate the optimal combination. In this section, we investigate the behavior of PSO, IDPSO_MC and IDPSO_SC in different levels of their parameters and are going to find the optimal level of these parameters and operators. The full factorial design is a conventional statistical method used for calibration of parameters. This method evaluates all the possible combinations of factors. As it would be explained later, there are one 3 level factors, two 5 level factors and one 4 level factors for PSO, and four 3 level factors and one 5 level factors for both IDPSO_MC, and IDPSO_SC. Moreover, there are 50 large test problems; each one will be run ten times. Hence,

```

Initialization of positions and velocities of swarm;
Iteration = 2; //  $\log_{10}(1)$  equal to 0
Do{
    Evaluate fitness of each particle;
    If Pbest Updated, then Apply the mutation operator on  $x_i^t$  ;
    Update Gbest;
    Apply crossover on velocities;
    Select randomly Reproduct_rate% of Population ;
    For selected particles
        Apply the mutation operator on these particles;

     $w = \frac{\omega}{\log_{10}^{iteration}}$  ;
    iteration = iteration + 1;
    Update velocity; //  $V_{ij}^{t+1} = wV_{ij}^t + C_1r_{1j}^t(Gbest - X_{ij}^t) + C_2r_{2j}^t(Pbest - X_{ij}^t)$  ;
    Update position; //  $X_{ij}^{t+1} = X_{ij}^t + V_{ij}^{t+1}$  ;
}while(stopping criterion is not met);

```

Fig. 5 Pseudo code of IDPSO_MC

the total number of replications is 312,500. In this condition, to reduce the number of experiments and to be economic, several experimental design techniques have been proposed. Taguchi optimization technique is one of them that have been successfully applied within recent years (Hajiaghahi-Keshteli 2010). Taguchi uses an orthogonal array to organize the experimental results. To select the appropriate orthogonal array it is necessary to compute total degree of freedom. In calibration of PSO parameters, the proper array should contain a degree of freedom for the total mean, two degrees of freedom for each factor with three levels, two degrees of freedom for the factor with four levels and four degree of freedom for the two factors with five levels. Thus, sum of the required degrees of freedom is $1 + 2 \times 1 + 3 \times 1 + 4 \times 2 = 14$. Therefore, the appropriate array must have at least 14 rows. The selected orthogonal array should be able to accommodate the factor level combinations in the experiment. Therefore, L25 (5^6) is an appropriate array satisfying these conditions. Using the dummy level technique we convert a five-level column into a three-level and four-level column. To assign the two-level factor to the three-level and four-level column from the orthogonal array L25, some of these levels are required to be replicated twice. In this research, third levels are chosen to be replicated twice. The appropriate orthogonal array for IDPSO_MC and IDPSO_SC is similarly L18 and L18, respectively. In the Taguchi method, the values of quality characteristics obtained through the experiments are transformed into a measure called signal-to-noise ratio (S/N ratio) which have three classes: The smaller-the-better, the larger-the-better, and nominal-is-best. Since the type of performance in this paper is the minimization, the corresponding smaller-the-better S/N ratio is used as follows:

$$S/N \text{ Ratio} = 10 \log_{10}(\text{Objective Function})^2$$

The minimum variances of objective function resulted from S/N ratio is the optimal operator combination, which explains the reason why a parameter design is also called robust design. The parameters and operators of algorithms and their levels are depicted in Table 2.

Data generation and calibration of parameters

Data generation

To compare our algorithm with other methods, some test problems are needed. In this regard, we generate the required data including the number of jobs (n), number of processors (m), range of processing times (p), size of jobs (s_j), range of earliness penalties (α_i) and tardiness penalties (β_i). In order to determine the number of jobs and number of processors, 6 levels for the number of jobs i.e. $n \in \{15, 20, 30, 50, 100, 150\}$ and 6 levels for the number of processors i.e. $m \in \{3, 5, 10, 15, 20, 30\}$ are considered. Selecting from sets of job numbers and processor numbers, 10 random test problems are produced as follows. The processing times and size of jobs are generated from uniform distributions on $[1, 30]$ and $[1, 10]$ respectively and *Cap* is set 10 in all instances. Moreover, we consider $n \in \{3, 4, 5, 7, 8\}$ and $m \in \{3, 4\}$ for small problems. The processing times and size of jobs are generated from uniform distributions between $[1, 15]$ and $[1, 10]$ respectively. Penalty of earliness and tardiness are uniform distribution over intervals (1,4) and (4,8) respectively. The due dates in Tavakkoli-Moghaddam et al.

Table 2 Factors and factor levels of algorithms

Factors	Number of levels	IDPSO_SC symbols	Number of levels	IDPSO_MC symbols	Number of levels	PSO symbols
w/ω	3	A(0)=1.3 A(1)=1.5 A(2)=1.7	3	A(0)=1.4 A(1)=1.5 A(2)=1.6	5	A(0)=1.3 A(1)=1.4 A(2)=1.5 A(3)=1.6 A(4)=1.7
<i>Reproduce_cross</i>	3	B(0)=0.3 × PopSize B(1)=0.4 × PopSize B(2)=0.5 × PopSize		–		–
<i>Reproduct_rate%</i>		–	3	B(0)=0.1 × PopSize B(1)=0.2 × PopSize B(2)=0.3 × PopSize		–
Population size	3	C(0)= $\frac{2}{3} \times n$ C(1)= n C(2)= $2 \times n$	3	C(0)= $\frac{n}{2}$ C(1)= n C(2)= $2 \times n$	4	C(0)= $n/2$ C(1)= $\frac{2}{3} \times n$ C(2)= n C(3)= $\times n$
(v_{min}, v_{max})	3	D(0)=(-3, 3) D(1)=(-3.5, 3.5) D(2)=(-4, 4)	3	D(0)=(-3, 3) D(1)=(-3.5, 3.5) D(2)=(-4, 4)	3	D(0)=(-3, 3) D(1)=(-3.5, 3.5) D(2)=(-4, 4)
C_1^*	5	E(0)=1 E(1)=1.3 E(2)=1.4 E(3)=1.5 E(4)=1.6	5	E(0)=1 E(1)=1.3 E(2)=1.4 E(3)=1.5 E(4)=1.6	5	B(0)=1.1 B(1)=1.2 B(2)=1.3 B(3)=1.4 B(4)=1.5

* $C_1 = C_2$

(2007) are generated from a uniform distribution. We use such procedure with some modifications to adapt the procedure for our problem as follows.

$$\bar{P} = \frac{\sum_{i=1}^m \sum_{j=1}^n P_{ij}}{m \times n} \quad (42)$$

$$\bar{B} = \frac{\sum_{j=1}^n S_j}{0.8 \times Cap} \quad (43)$$

$$P = (m + \bar{B} - 1) \times \bar{P} \quad (44)$$

The due dates are generated from following distribution.

$$U \sim (\text{Floor of } (1 - L) \times P, \text{ Roun of } (1 + H) \times P) \quad (45)$$

where L and H are lower and upper limits and are set to be 0.1 and 0.7 respectively. Each size of the large test problem includes 5 generated instances (i1~i5), resulting in $5 + 10 \times 5 = 55$ test problems. Due to stochastic nature of metaheuristics, ten replications were performed for each trial to achieve more reliable results. Hence, we have 550 results for each algorithm. The algorithms are coded in C# (visual

studio 2008) and run on a Pentium IV PC with 2.6GHz and 256MB of RAM memory.

Parameters calibration

As mentioned earlier, the experiments on the PSO, IDPSO_MC and IDPSO_SC were based on the L25, L18 and L18 orthogonal array respectively. These algorithms are coded in C sharp and run on a Pentium IV PC with 2.6GHz and 256MB RAM of memory. To be fair, the stopping criterion for all algorithms is set to $2.9 \times 10^{-3} \times m^2 n$ seconds. Results are individually transformed into S/N ratios, and then these ratios of trials are averaged in each level and are shown in Figs. 6, 7 and 8. The obtained levels of parameters are shown in Table 3. For example, let us consider factor **Reproduce_Cross** for IDPSO_SC algorithm. As shown by Table 2, there are three levels of **Reproduce_Cross** for IDPSO_SC, i.e., B(0), B(1), and B(2). The obtained S/N ratios are depicted in Fig. 6. For this example, we just need the second part of figure {B(0), B(1), and B(2)}. In order to minimize the variation effects, we should select maximum S/N ratio among B(0), B(1), and B(2). As can be seen, B(1)

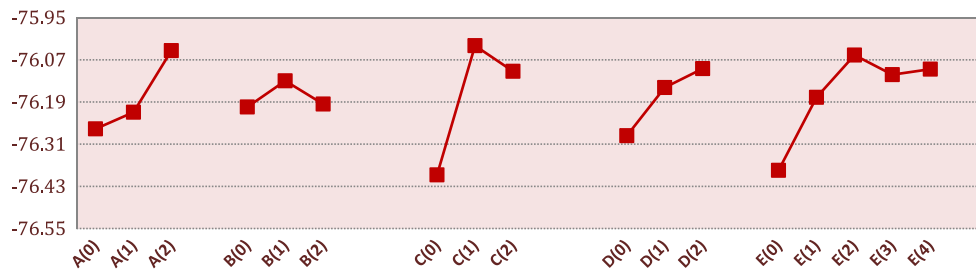


Fig. 6 Mean S/N ratio plot for each level of the factors in IDPSO_SC

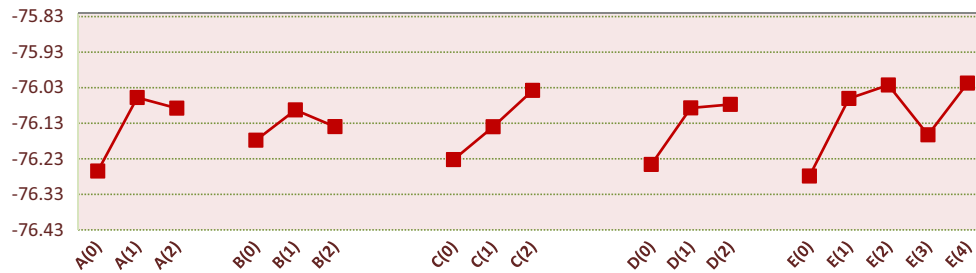


Fig. 7 Mean S/N ratio plot for each level of the factors in IDPSO_MC

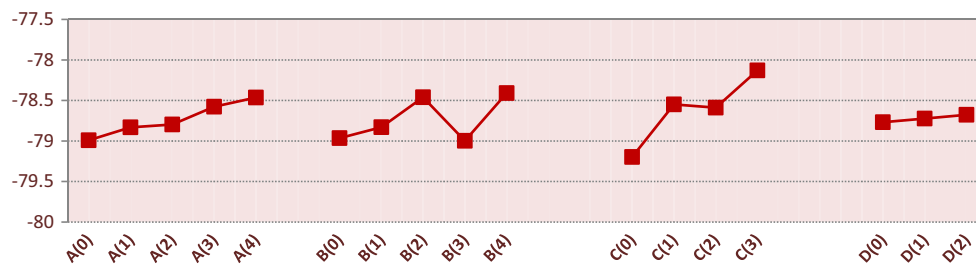


Fig. 8 Mean S/N ratio plot for each level of the factors in PSO

is maximum. From Table 2, we can see that $B(1) = 0.4 \times \text{PopSize}$. Table 3 summarizes the results.

Computational experiments

In this section, we present the results of proposed IDPSO_MC and IDPSO_SC versus a pure PSO, PSO_{vns} of Tasgetiren et al. (2007) as a well-known algorithm of PSO literature, SA of Manjeshwar et al. (2009), and VNS proposed by Lei and Guo (2011), and EDD dispatching rule as a well-known heuristic. A fair analysis is that we compare the methods at their best outputs for a specific problem. Hence, we calibrate all of the reference algorithms (SA, GA, VNS, and PSO_{vns}) with Taguchi and the results are employed for comparisons. In order to evaluate the performance of algorithms against exact solution, a commercial solver LINGO is used to solve the problems. To this end, we execute the LINGO for small instances and the outputs are presented in Table 4. In this table, the first columns represent the adapted algorithms and the remaining columns show the performance

of algorithms on small problem instances. To make the results reliable, five replications are implemented for all of the metaheuristics and the average of solutions are reported in Table 4. Lingo is also implemented for both P_1 and P_2 models within 300 seconds time limit and the results show that it can give optimal solution for all of the small problems. As shown by Table 4, GA, VNS, PSO and PSO_{vns} show relatively identical performance, while SA have poorest results, and IDPSO_SC and IDPSO_MC gives optimal solutions.

We then present the results of our proposed IDPSO_MC and IDPSO_SC and compared them versus basic PSO, PSO_{vns} , SA, GA, VNS and EDD rule. Tables 5, 6, 7, 8 and 9 illustrates the best, worst and averaged results obtained from algorithms in each instances for ten replications. The notation $i1 \sim i5$ indicates the instance number for a problem. In approximately all of the cases, IDPSO_SC and IDPSO_MC produced the best solution. IDPSO_SC and IDPSO_MC produce smaller value of min and average than other algorithms on all 50 instances except $i4$ and $i5$ of 15j3m, $i1$ of 20j5m, $i5$ of 100j15m, $i3$ of 150j20m and $i5$ of 150j30m.

Table 3 The obtained parameters for PSOs

Factors	IDPSO_SC Symbols	IDPSO_MC Symbols	PSO symbols
$w(\omega)$	A(2)=1.7	A(1)=1.5	A(4)=1.7
<i>Reproduce_Cross</i>	B(1)=0.4 PopSize	–	–
<i>Reproduct_rate</i> %	–	B(1)=0.2× PopSize	–
<i>inloop</i>	–	–	–
<i>Population size</i>	C(1) = n	C(2) = $2 \times n$	C(3) = $2 \times n$
(v_{min}, v_{max})	(−4, 4)	(−4, 4)	(−4, 4)
C_1	E(2)=1.4	E(4)=1.6	B(4)=1.5

Table 4 Comparison of algorithms in small instances

Problem	Algorithms							
	SA	GA	VNS	PSO	PSO _{vns}	IDPSO_SC	IDPSO_MC	Lingo Model Model
3j3m	206.71	203.61	204.16	203.05	200.00	200.00	200.00	200.00 200.00
4j3m	315.12	313.46	311.50	317.15	310.39	309.00	309.00	309.00 309.00
7j3m	166.07	152.84	153.19	151.97	152.04	151.00	151.00	151.00 151.00
5j4m	198.06	193.49	194.53	192.49	193.67	191.00	191.00	191.00 191.00
8j4m	298.10	295.45	293.54	294.28	293.46	289.00	289.00	289.00 289.00
Average	236.81	231.77	231.38	231.79	229.91	228.00	228.00	228.00 228.00

Table 5 The best, worst and average obtained results from algorithms in 15j3m and 20j5m test problem

Problem	C_{max}	15j3m					20j5m				
		i1	i2	i3	i4	i5	i1	i2	i3	i4	i5
IDPSO_MC	Best	842	646	1654	806	1172	1624	1220	1787	3528	1118
	Average	848	654.7	1699.5	811.5	1192.7	1690.7	1405	1837.5	3666.2	1224.5
	Worst	866	671	1754	828	1214	1810	1855	1884	3808	1298
IDPSO_SC	Best	842	646	1718	806	1172	1577	1220	1770	3527	1254
	Average	867.2	653.5	1744	841.7	1182.5	1622.2	1246.2	1809.7	3668.2	1325
	Worst	943	676	1776	891	1214	1667	1305	1860	3731	1368
PSO _{vns}	Best	1021	656	1671	825	1172	1897	1855	1946	3666	1452
	Average	1021	825.2	1727.5	904	1253.7	1897.5	1917.2	1946	3829.7	1496
	Worst	1021	987	1785	931	1457	1899	1963	1946	4015	1541
GA	Best	942	746	1771	906	1269	1782	1463	1928	3618	1336
	Average	1108	822	1821.2	939	1393.5	1899.7	1709.2	2177.7	3968.2	1572.5
	Worst	1318	1020	1902	1019	1610	2126	1836	2406	4403	1695
PSO	Best	842	646	1671	806	1172	1682	1363	1828	3518	1236
	Average	1008	722	1721.2	839	1293.5	1799.7	1609.2	2077.7	3868.2	1472.5
	Worst	1218	920	1802	919	1510	2026	1736	2306	4303	1595
SA	Best	875	743	1743	823	1163	1834	1523	1873	3418	1518
	Average	933	767.2	1806.5	921.5	1231.5	1946.5	1662	2029.5	3760.2	1572.5
	Worst	973	802	1831	1045	1272	2235	1724	2215	4071	1643
VNS	Best	973	656	1616	806	1137	1635	1363	1828	3712	1412
	Average	992	717.7	1690.7	846.7	1177.7	1688	1426	1981.7	3956.7	1523.5
	Worst	1021	847	1729	931	1265	1753	1472	2106	4303	1595

Bold values indicate the best obtained results for a test problem

Table 6 The best, worst and average results obtained from algorithms in 30j5m and 30j10m test problem

Problem	C_{max}	30j5m					30j10m				
		i1	i2	i3	i4	i5	i1	i2	i3	i4	i5
IDPSO_MC	Best	2105	3634	3403	3822	7941	3260	3021	3461	2196	3416
	Average	2229	3876	3614	4338.2	8149	3394.5	3337	4017	2550.5	3561.7
	Worst	2536	4154	3786	5048	8250	3694	3667	4988	2934	3750
IDPSO_SC	Best	2069	3667	3161	4016	8291	3209	3008	3423	2008	3440
	Average	2265.2	3901.7	3445.5	4267.5	8322	3461.7	3081.2	3531	2422.2	3651.2
	Worst	2471	4202	3573	4486	8356	4086	3135	3638	2698	3920
PSO _{vns}	Best	2582	4729	4313	4939	8175	4411	4466	4512	3108	4169
	Average	2635.2	5133.5	4456	5066	8249.2	4522.7	4610	4630	3400	4210.2
	Worst	2655	5387	4622	5199	8380	4655	4754	4733	3722	4224
GA	Best	2382	3965	3610	4646	8269	4239	4164	4128	2634	3723
	Average	2725.2	4490.2	3789	4827.5	8619.5	4623.2	5025.2	4501.7	3044	3958.5
	Worst	3092	5434	4036	5177	8939	5348	5744	5092	3666	4149
PSO	Best	2364	3865	3589	4546	8169	3839	3764	3772	2334	3623
	Average	2733.2	4433	3726.5	4727.5	8519.5	4223.2	4625.2	4151.7	2744	3808.5
	Worst	3192	5334	3936	5077	8839	4948	5344	4792	3366	3914
SA	Best	2557	4256	4175	4983	9264	4808	4996	4813	2961	4287
	Average	2771.7	4571	4595	5115.7	9553.7	5211.2	5574.5	5089.5	3507.2	4443.2
	Worst	2918	4860	4965	5274	9719	5748	6172	5426	4017	4681
VNS	Best	2854	3865	3752	4763	8537	4672	4466	4753	3687	4042
	Average	2950.2	3933.5	3880.5	4894.7	8705.2	4752.2	4751.5	4808	3817.2	4218.5
	Worst	3015	4134	3972	5077	8911	4816	4976	4832	3974	4356

Bold values indicate the best obtained results for a test problem

Table 7 The best, worst and average results obtained from algorithms in 50j10m and 50j15m test problem

Problem	C_{max}	50j10m					50j15m				
		i1	i2	i3	i4	i5	i1	i2	i3	i4	i5
IDPSO_MC	Best	8058	7594	8943	11,549	10,672	6012	9738	7671	5863	4875
	Average	8681	7659	9751.2	11,867.5	11,498.7	7004	10,355.2	8594.2	6466	5497.7
	Worst	9115	7771	10,746	12,251	12,336	7767	11,293	9301	7376	6035
IDPSO_SC	Best	8557	8101	9361	11,100	11,113	6024	9679	8024	5863	5492
	Average	9176.5	8685.5	10,604.2	12,265.2	11,524.7	6565.2	10,251.2	8160.7	6144	5717.7
	Worst	9821	9360	11,620	13,179	11,896	7336	10,476	8400	6520	5868
PSO _{vns}	Best	10,031	12,503	10,494	16,172	12,435	7479	12,593	9921	7096	6825
	Average	10,520.7	12,917.2	10,494	16,172	12,910.7	8013.2	13,427	10,545.2	7785.5	7595
	Worst	10,949	13,135	10,494	16,172	13,700	8402	14,651	11,351	8344	8628
GA	Best	10,868	8767	12,211	12,998	11,225	7509	11,564	9734	7960	7576
	Average	11,464.2	10,944.5	14,171.5	14,057.5	13,889.7	9169.7	12,716.5	10,203.2	9064.5	8192
	Worst	12,141	13,463	15,754	15,247	15,914	9942	13,873	10,812	10,171	8770
PSO	Best	10,768	8667	12,111	12,898	11,125	7809	13,165	10,334	8060	7476
	Average	11,364.2	10,844.5	14,071.5	13,957.5	13,789.7	10,169.7	14,241.5	10,628.2	9864.5	8292
	Worst	12,041	13,363	15,654	15,147	15,814	11,842	14,773	11,044	11,989	8970
SA	Best	11,367	12,176	9430	16,438	13,931	7809	13,165	9337	8338	8037
	Average	11,644.7	12,573.2	10,215.7	17,138.7	15,724	8602.25	13,834.5	10,057	9571.7	8352.5
	Worst	11,842	13,765	10,629	17,954	17,492	9731	14,371	10,841	10,428	8790
VNS	Best	10,031	12,896	10,494	16,231	12,435	8821	12,871	9971	7830	7185
	Average	10,198	12,986.5	10,654.7	16,288	13,270.2	8901.2	13,238.7	10,210.5	8018.7	7595
	Worst	10,326	13,125	11,043	16,375	13,762	9032	13,652	10,593	8351	8126

Bold values indicate the best obtained results for a test problem

Table 8 The best, worst and average results obtained from algorithms in 100j15m and 100j20m test problem

Problem	C_{max}	100j15m					100j20m				
		i1	i2	i3	i4	i5	i1	i2	i3	i4	i5
IDPSO_MC	Best	59,279	48,134	48,221	54,223	48,017	55,791	52,741	46,156	45,463	50,359
	Average	61,650.7	51,295.5	52,210.5	58,914.5	48,706.2	59,284.5	57,454.2	48,412.7	46,456.2	51,308
	Worst	63,080	54,737	56,376	61,692	49,684	61,239	62,109	52,029	48,880	53,560
IDPSO_SC	Best	57,363	51,536	50,838	57,265	44,167	55,502	51,477	41,753	44,262	47,553
	Average	59,697.2	52,404.7	53,535.5	57,989.7	49,270.5	58,420.5	55,138.5	44,820.5	46,165.5	48,863.2
	Worst	61,558	52,923	58,595	58,916	53,099	63,792	59,806	47,592	48,187	50,891
PSO _{uns}	Best	71,845	58,318	59,397	61,379	38,637	57,449	67,210	50,425	48,924	49,228
	Average	72,642.2	61,840.5	60,134	63,881	41,793.2	58,248	68,201	52,557	50,421.5	52,528.2
	Worst	73,468	64,788	61,701	65,744	45,638	59,628	69,043	54,738	52,700	54,887
GA	Best	66,051	57,934	59,745	60,165	48,074	63,965	65,149	59,252	59,802	61,812
	Average	70,428.5	61,161.7	65,837	66,627.2	58,863.7	69,137.5	72,897.2	62,646.7	63,485.2	64,672.5
	Worst	75,682	64,759	69,782	74,120	66,729	80,517	76,423	66,555	68,262	70,315
PSO	Best	61,951	53,205	57,645	60,065	47,974	61,723	67,109	53,863	48,183	49,265
	Average	65,578.5	58,961.7	64,737	65,877.2	57,013.7	63,809.7	68,227.7	57,618.7	50,642.2	52,423
	Worst	69,582	64,659	67,682	74,020	66,629	67,387	69,342	61,221	52,223	54,187
SA	Best	80,738	73,176	69,757	68,261	69,281	75,581	78,173	71,653	71,762	74,174
	Average	82,238.5	74,343	73,165.7	71,098.2	70,243.2	78,231.2	87,551.5	74,926.2	74,671.5	74,899
	Worst	84,861	75,642	74,672	77,519	71,522	80,620	91,753	78,763	80,245	76,517
VNS	Best	63,072	57,541	67,134	65,743	53,657	63,965	69,763	59,637	58,875	61,765
	Average	66,711.5	59,992	67,706.7	67,910.2	56,501.2	65,388.5	71,947.5	61,999.2	61,091.5	63,048.7
	Worst	70,431	64,659	68,437	70,237	60,137	67,517	75,738	65,726	63,623	64,342

Bold values indicate the best obtained results for a test problem

Table 9 The best, worst and average results obtained from algorithms in 150j20m and 150j30m test problem

Problem	C_{max}	150j20m					150j30m				
		i1	i2	i3	i4	i5	i1	i2	i3	i4	i5
IDPSO_MC	Best	188,336	144,899	169,797	147,092	189,307	93,357	101,990	102,044	97,672	85,498
	Average	195,364.2	157,041.7	180,290.7	152,460.7	194,523	96,663.7	105,246.2	107,728.2	100,077.7	89,969.5
	Worst	199,944	166,829	196,496	158,775	197,515	99,334	111,592	113,895	102,713	92,909
IDPSO_SC	Best	183,435	148,967	158,451	139,099	182,368	95,973	106,918	10,1003	93,280	77,880
	Average	193,659.2	154,512	162,577.2	142,266.7	191,422.2	97,815.5	111,264.5	105,466	95,088	78,953.5
	Worst	207,219	157,381	167,887	144,826	198,963	100,112	118,312	110,334	97,257	79,523
PSO _{uns}	Best	210,511	163,870	147,318	154,580	212,128	106,070	108,049	108,711	101,525	105,239
	Average	215,107.5	166,641	151,259.7	156,450.2	213,109	115,195	110,683	116,763.5	107,583.2	107,735.5
	Worst	219,901	169,541	154,349	157,077	214,320	126,407	116,431	122,221	118,299	111,819
GA	Best	185,539	169,417	173,112	159,369	201,095	112,186	136,427	123,808	101,524	93,051
	Average	199,291.7	177,573.2	184,330	164,810.7	210,194.7	129,091.5	144,679.2	130,264	108,809.7	99,669.5
	Worst	205,463	186,264	194,437	174,489	223,775	145,780	158,343	134,161	114,337	105,753
PSO	Best	235,439	179,317	183,012	159,269	200,995	102,086	136,327	112,900	101,145	92,951
	Average	248,566.7	189,723.2	189,230	164,710.7	212,594.7	108,991.5	139,579.2	125,164	106,209.7	7069.5
	Worst	259,779	196,164	194,337	174,389	223,675	115,680	146,889	134,061	114,237	101,904
SA	Best	214,356	203,300	207,734	191,242	241,314	134,623	163,712	146,104	101,828	101,661
	Average	230,988.8	213,087.3	221,195.5	197,772.3	252,233.5	152,409.5	166,114.5	151,316.5	118,071.3	112,103
	Worst	242,435	223,516	233,324	209,386	268,530	156,421	125,647	126,641	117,856	107,645
VNS	Best	246,731	181,645	183,012	161,875	201,645	134,526	107,487	115,273	1024,54	105,345
	Average	254,820	192,418.5	190,245.8	167,751.3	210,677.8	143,277.8	116,334.5	118,855.3	111,949.5	106,989.8
	Worst	259,825	198,764	203,141	174,253	216,547	156,421	125,647	126,641	117,856	107,645

Bold values indicate the best obtained results for a test problem

Table 10 Results of EDD on test problems

Problem	EDD				
	i1	i2	i3	i4	i5
15j3m	1265	1236	2882	1223	2203
20j5m	2229	2522	2295	5528	2177
30j5m	3695	7243	5707	6146	12,389
30j10m	7217	5612	5771	5392	4871
50j10m	12,384	13,684	11,301	17,000	15,181
50j15m	10,372	16,709	15,483	10,464	9653
100j15m	83,435	75,642	76,779	77,519	73,458
100j20m	81,311	79,145	69,235	68,879	74,376
150j20m	278,472	208,666	197,016	179,299	223,399
150j30m	179,307	133,084	125,447	107,755	91,220

Table 10 shows results obtained of EDD on test problems in which the first column represents the data sets characteristics and the remaining columns show results of EDD on instance i1~i5 based on C_{max} .

Table 11 Improvements of algorithms in comparison with EDD

Problem	SA	VNS	GA	PSO	PSO _{vns}	IDPSO_MC	IDPSO_SC
	Improvement %	Improvement %	Improvement %	Improvement %	Improvement %	Improvement %	Improvement %
<i>15j3m</i>							
i1	26.245	21.58	12.41	20.31	19.28	32.96	31.44
i2	37.92	41.92	33.49	41.58	33.23	47.02	47.12
i3	37.31	41.33	36.80	40.27	40.05	41.03	39.48
i4	24.65	30.76	23.22	31.39	26.08	33.64	31.17
I5	44.09	46.53	36.745	41.28	43.08	45.85	46.32
<i>20j5m</i>							
i1	12.67	24.27	14.77	19.25	14.87	24.14	27.22
i2	34.09	43.45	32.22	36.19	23.97	44.29	50.58
i3	11.56	13.64	5.10	9.46	15.20	19.93	21.14
i4	31.97	28.42	28.21	30.02	30.72	33.67	33.64
I5	27.76	30.01	27.76	32.36	31.28	43.75	39.13
<i>30j5m</i>							
i1	24.98	20.15	26.24	26.02	28.68	39.67	38.69
i2	36.89	45.69	38.00	38.79	29.12	46.48	46.13
i3	19.48	32.00	33.60	34.70	21.92	36.67	39.62
i4	16.76	20.35	21.45	23.08	17.57	29.41	30.56
I5	22.88	29.73	30.42	31.23	33.41	34.22	32.82
<i>30j10m</i>							
i1	27.79	34.15	35.93	41.48	37.33	52.96	52.03
i2	0.66	15.33	10.45	17.58	17.85	40.53	45.09
i3	11.80	16.68	21.99	28.05	19.77	30.39	38.81
i4	34.95	29.20	43.54	49.10	36.94	52.69	55.07
I5	8.78	13.39	18.73	21.81	13.56	26.87	25.04
<i>50j10m</i>							
i1	5.96	17.65	7.42	8.23	15.04	29.90	25.90
i2	8.11	5.09	20.01	20.75	5.60	44.02	36.52

On the basis of the foregoing computational results of EDD, we calculated the improvement's percentage (**Improvement %**) of each algorithm in comparison with it. Table 11 shows the improvements' percentage of each algorithm against EDD results. These tables demonstrated the robust performance of IDPSO_SC and IDPSO_MC. Among algorithms, SA has smaller improvement and even in some instances has worst performance. Other algorithms have improvement, but comparing to IDPSO_MC and IDPSO_SC, their improvements are relatively small. Furthermore, PSO_{vns} have more improvement among algorithms of BP literature; and VNS has better improvement than SA and GA.

In order to make the comparison easy and comprehensive, we use the relative percentage deviation (*RPD*) as a common performance measure of as follows.

$$RPD = \frac{Alg_{sol} - Min_{sol}}{Min_{sol}} \times 100,$$

Table 11 continued

Problem	SA	VNS	GA	PSO	PSO _{vns}	IDPSO_MC	IDPSO_SC
	Improvement%	Improvement %	Improvement %	Improvement %	Improvement %	Improvement %	Improvement %
i3	9.60	5.71	−25.40	−24.51	7.14	13.71	6.16
i4	−0.81	4.18	17.30	17.89	4.87	30.19	27.85
I5	−3.57	12.58	8.50	9.16	14.95	24.25	24.08
<i>50j15m</i>							
i1	17.06	14.18	11.59	1.94	22.74	32.47	36.70
i2	17.20	20.76	23.89	14.76	19.64	38.02	38.64
i3	35.04	34.05	34.10	31.35	31.89	44.49	47.29
i4	8.52	23.36	13.37	5.72	25.59	38.20	41.28
I5	13.47	21.31	15.13	14.09	21.31	43.04	40.76
<i>100j15m</i>							
i1	1.434	20.04	15.58	21.40	9.68	23.35	25.78
i2	1.717	20.68	19.14	22.05	12.45	27.38	25.81
i3	4.70	11.81	14.25	15.68	17.37	28.26	26.44
i4	8.28	12.39	14.05	15.01	14.27	20.94	22.18
I5	4.37	23.08	19.86	22.38	40.68	30.87	30.07
<i>100j20m</i>							
i1	3.787	19.58	14.97	21.52	11.79	10.22	11.53
i2	−10.62	9.09	7.894	13.79	3.51	18.71	21.99
i3	−8.22	10.45	9.5151	16.77	14.54	21.28	27.12
i4	−8.40	11.30	7.83	26.47	7.50	14.77	15.31
I5	−0.70	15.22	13.04	29.51	10.40	12.48	16.65
<i>150j20m</i>							
i1	17.05	8.49	28.43	10.73	5.84	29.84	30.45
i2	−2.11	7.78	14.90	9.07	6.73	24.74	25.95
i3	−12.27	3.43	6.439	3.95	9.43	8.48	17.48
i4	−10.30	6.44	8.08	8.13	1.78	14.96	20.65
I5	−12.90	5.69	5.91	4.83	4.60	12.92	14.31
<i>150j30m</i>							
i1	15.00	20.09	28.00	39.21	−5.38	46.09	10.51
i2	−24.81	12.58	−8.71	−4.88	16.83	20.91	16.39
i3	−20.62	5.25	−3.83	0.22	6.92	14.12	15.92
i4	−9.57	−3.89	−0.97	1.43	0.15	7.12	11.75
I5	−22.89	−17.28	−9.26	−6.41	−18.10	1.37	13.44

Bold values indicate the best obtained results for a test problem

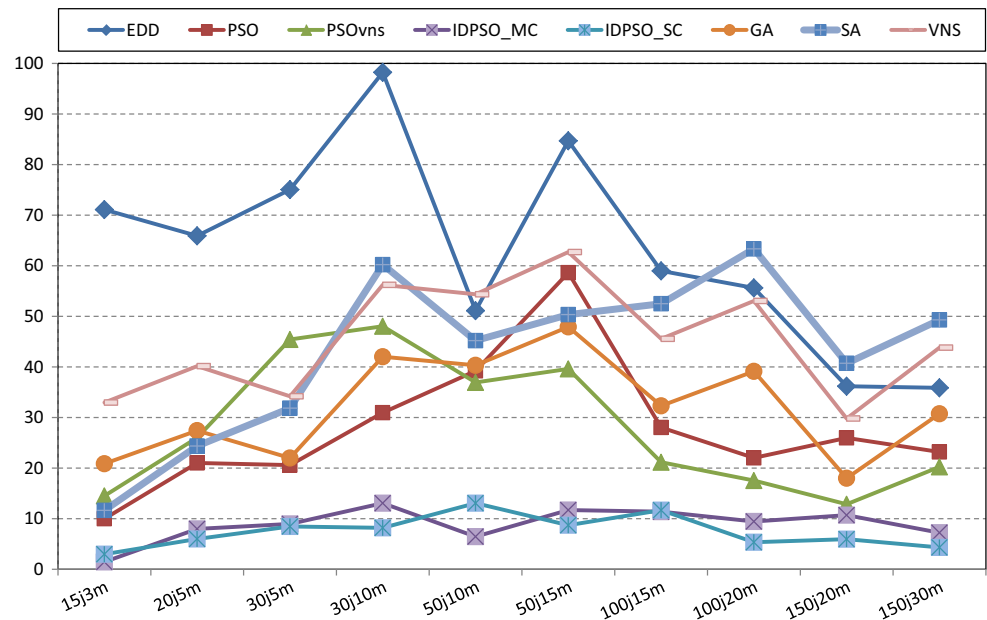
where Alg_{sol} and Min_{sol} are the obtained objective value and the obtained best solution respectively for each replication of instance in a given test problem. The results of the experiments were transformed into the RPD, averaged for test problems (5 data points per average for EDD and 50 data points per average for other algorithms) and are shown in Table 12. According to the results, the average RPD obtained by suggested IDPSO_MC and IDPSO_SC are 7.475831 and 8.845512 respectively, while those of SA, VNS, GA, PSO and PSO_{vns} are 42.94979, 30.00181, 32.07578, 27.96739 and 26.24862. The IDPSO_SC and IDPSO_MC can find the better solution than other algorithms for all the instances. The SA

with a RPD of 42.94979% has the worst performance among metaheuristics. As it obvious, results of PSO are superior to SA, VNS and GA.

To analyze the interaction between quality of the algorithms and different size of problems, the average RPDs obtained by each algorithm are shown in Fig. 9. As it can be seen, IDPSO_SC and IDPSO_MC keep their robust performance in all ranges of problem sizes. In first three sizes of problems, VNS has the better performance in comparison with PSO_{vns}, but by increasing the size, PSO_{vns} outperforms VNS. In most sizes, VNS and GA are competitive. SA has only good performance in 15j3m and 20j5m, and, its perfor-

Table 12 Average *RPD* for the algorithms

Problem	Algorithms							
	EDD	SA	VNS	GA	PSO	PSO _{vns}	IDPSO_MC	IDPSO_SC
15j3m	71.10386	11.64077	7.337974	20.86969	10.00135	14.51688	1.454169	2.986509
20j5m	65.91682	24.37201	16.92578	27.41133	21.0164	26.0181	7.986108	6.010575
30j5m	75.05277	31.85491	22.25766	22	20.59167	29.20591	8.970017	8.461515
30j10m	98.23645	60.22765	52.02189	42.02474	30.96092	44.40652	13.07099	8.196655
50j10m	51.13031	45.21073	37.55866	40.33044	39.22765	36.93511	6.457301	13.06361
50j15m	84.69532	50.34244	42.10098	47.91267	58.63781	39.61298	11.71638	8.715819
100j15m	58.98193	52.49404	30.56408	32.32027	28.02467	21.15948	11.40593	11.68627
100j20m	55.61856	63.33839	35.33569	39.13034	22.03273	17.53808	9.446156	5.354781
150j20m	36.1904	40.72452	27.3944	18.01219	25.97575	12.85551	10.70729	5.961649
150j30m	35.88024	49.29243	28.52094	30.74617	23.20499	20.23765	7.240788	4.320925
<i>RPD</i>	63.28067	42.94979	30.00181	32.07578	27.96739	26.24862	8.845512	7.475831

Fig. 9 Means plot for the interaction among algorithms

mance becomes worse when size increases. The performance of EDD is better than SA in later three sizes. After PSO_{vns}, VNS has the best behavior among other metaheuristics. From 15j3m to 30j10m problems, PSO outperforms PSO_{vns}, but, the performance of PSO_{vns} improves significantly when size increases. Based on the results, we conclude that the proposed IDPSO_MC and IDPSO_SC can be used to for effective solving of the flow shops with BPs.

Conclusion and insights

The batch processing can be an advantage for companies that produce a wide range of items and different version of a basic product. It is cheaper to produce a number of each

item because processors are utilized more effectively; the materials are procured in bulk and the human resources can specialize in own task. Then, when our process is characterized by fluctuating demand, short production runs of a wide variety of products, batch processing is an appropriate manufacturing system. A flow shop scheduling problem with batch processors and earliness/tardiness criteria was studied in current paper. In manufacturing environment, jobs which are completed before their due dates may cause earliness penalties such as deterioration of items, and opportunity and holding costs for finished items while jobs which are late incur a tardiness penalty. Therefore, an ideal schedule is one in which all jobs are completed exactly on due dates. Neither earliness nor tardiness is desirable. The earliness represents management concerns and the tardiness represents the customer concerns. So, both earliness and

tardiness are important and should be considered in manufacturing planning. In our paper, to solve the problem addressed, PSO was designed by using such a chaotic map instead of random parameters used in optimization algorithms. As a novel application of chaos, a logistic chaotic map was incorporated into PSO to improve its global convergence by escaping from local results. We also customized PSO algorithm and two most important and dangerous properties of PSO have been investigated. As a result, some suggestion and modifications have been incorporated into basic PSO. As experimental results show, the performance of the PSO algorithm was significantly improved by our modifications. Moreover, since the appropriate selection of parameters has the significant impact on the efficiency of PSO; we adopted Taguchi experimental design as a systematic technique for tuning parameters. Afterwards, in order to verify the developed model and evaluate the performance of our algorithms against exact methods, a commercial optimization solver LINGO 11.0 has been used to solve the problem instances in small size, then to evaluate the performances of the proposed algorithms, an extensive computational analysis has been carried out with a total of 50 large instances. The performance of the suggested algorithms has been compared with a basic PSO, PSO_{vns} as a well-known algorithm from PSO literature, and GA, SA and VNS as effective algorithms from BP literature. The obtained results showed that (i) PSO has the better behavior than GA, SA and VNS of BP literature, and (ii) IDPSO_SC and IDPSO_MC have remarkable and robust performance for addressing BP problems.

References

- Alatas, B., Akin, E., & Ozer, A. B. (2009). Chaos embedded particle swarm optimization algorithms. *Chaos, Solitons and Fractals*, 40, 1715–1734.
- Chang, P. Y., Damodaran, P., & Melouk, S. (2004). Minimizing makespan on parallel batch processing machines. *International Journal of Production Research*, 42, 4211–4220.
- Cheng, M.-Y., Huang, K.-Y., & Chen, H.-M. (2012). K-means particle swarm optimization with embedded chaotic search for solving multidimensional problems. *Applied Mathematics and Computation*, 219, 3091–3099.
- Chou, D. F. (2007). A joint GA+DP approach for single burn-in oven scheduling problems with makespan criterion. *International Journal of Advanced Manufacturing Technology*, 35, 587–595.
- Chou, D. F., Chann, C. P., & Wang, M. H. (2006). A hybrid genetic algorithm to minimize makespan for the single batch machine dynamic scheduling problem. *International Journal of Advanced Manufacturing Technology*, 31, 350–359.
- Chou, D. F., & Wang, M. H. (2008). Scheduling for a single semiconductor batch processing machine to minimize total weighted tardiness. *Journal of the Chinese Institute of Industrial Engineers*, 25, 136–147.
- Damodaran, P., Hirani, N. S., Mario, C., & Gallego, V. (2009). Scheduling identical parallel batch processing machines to minimise makespan using genetic algorithms. *European Journal of Industrial Engineering*, 3, 187–206.
- Damodaran, P., Manjeshwar, P. K., & Srihari, K. (2006). Minimizing makespan on a batch processing machine with non-identical job sizes using genetic algorithms. *International Journal of Production Economics*, 103, 882–891.
- Damodaran, P., & Srihari, K. (2004). Mixed integer formulation to minimize makespan in a flow shop with batch processing machines. *Mathematical and Computer Modelling*, 40, 1465–1472.
- Damodaran, P., Srihari, K., & Lam, S. (2007). Scheduling a capacitated batch-processing machine to minimize makespan. *Robotics and Computer-Integrated Manufacturing*, 23, 208–16.
- Dauxois, T., Ruffo, S., Arimondo, S., & Wilkens, M. (2002). *Dynamics and thermodynamics of systems with long range interactions: An introduction* (Lect. Notes in Phys. Vol. 602, 1st ed.). Springer: Berlin.
- Gutierrez, J. M., & Iglesias, A. (1998). Mathematica package for analysis and control of chaos in nonlinear systems. *Computers in Physics*, 12(6), 608–619.
- Hajiaghahi-Keshteli, M. (2010). The allocation of customers to potential distribution centers in supply chain networks: GA and AIA approaches. *Applied Soft Computing*, 2, 2069–2078.
- He, Y., Qifa, X., Yang, S., & Liao, L. (2014). Reservoir flood control operation based on chaotic particle swarm optimization algorithm. *Applied Mathematical Modelling*, 38, 4480–4492.
- Husseinzadeh Kashan, A., & Karimi, B. (2007). Scheduling a single batch-processing machine with arbitrary job sizes and incompatible job families: An ant colony framework. *Journal of the Operational Research Society*, 59, 1269–1280.
- Husseinzadeh Kashan, A., Karimi, B., & Jenabi, M. (2008). A hybrid genetic heuristic for scheduling parallel batch processing machines with arbitrary job sizes. *Computers and Operations Research*, 35, 1084–1098.
- Husseinzadeh Kashan, A., Karimi, B., & Jolai, F. (2006). Effective hybrid genetic algorithm for minimizing makespan on a single-batch-processing machine with non-identical job sizes. *International Journal of Production Research*, 44, 2337–2360.
- Iqbal, S., Zang, X., Zhu, Y., Liu, X., & Zhao, J. (2014). *Introducing undergraduate electrical engineering students to chaotic dynamics: Computer simulations with logistic map and buck converter*, 2014 8th Asia Modelling Symposium (AMS 2014), Taipei, Taiwan; 09/2014.
- Kennedy, J., & Eberhart, R. C. (1997). A discrete binary version of the particle swarm algorithm. In *Proceedings of the world multiconference on systemics, cybernetics and informatics* (pp. 4104–9). Piscataway, NJ.
- Koh, S. G., Koo, P. H., Ha, J. W., & Lee, W. S. (2004). Scheduling parallel batch processing machines with arbitrary job sizes and incompatible job families. *International Journal of Production Research*, 42, 4091–4107.
- Kuru, L., Ozturk, A., Kurua, E., Kandar, & Kandar, O. (2015). Determination of voltage stability boundary values in electrical power systems by using the Chaotic Particle Swarm Optimization algorithm. *Electrical Power and Energy Systems*, 64, 873–879.
- Lauff, V., & Werner, F. (2004). Scheduling with common due date, earliness and tardiness penalties for multimachine problems: A survey. *Mathematical and Computer Modelling*, 40, 637–655.
- Lei, D., & Guo, X. (2011). Variable neighborhood search for minimizing tardiness objectives on flow shop with batch processing machines. *International Journal of Production Research*, 49, 519–529.
- Li, X., & Yin, M. (2013). An opposition-based differential evolution algorithm for permutation flow shop scheduling based on diversity measure. *Advances in Engineering Software*, 55, 10–31.
- Malve, S., & Uzsoy, R. (2007). A genetic algorithm for minimizing maximum lateness on parallel identical batch processing machines with dynamic job arrivals and incompatible job families. *Computers & Operations Research*, 34, 3016–3028.

- Manjeshwar, P. K., Damodaran, P., & Srihari, K. (2009). Minimizing makespan in a flow shop with two batch processing machines using simulated annealing. *Robotics and Computer-Integrated Manufacturing*, 25, 667–679.
- Mark Berliner, L. (1992). Statistics, probability and chaos. *Statistical Science*, 7(1), 69–122.
- May, R. M. (1976). Simple mathematical models with very complicated dynamics. *Nature*, 261, 459.
- Mazumdar, C. S., Mathirajan, M., Gopinath, R., & Sivakumar, A. I. (2008). Tabu search methods for scheduling a burn-in oven with non-identical job sizes and secondary resource constraints. *International Journal of Operational Research*, 3, 119–139.
- Melouk, S., Damodaran, P., & Chang, P. Y. (2004). Minimizing makespan for single machine batch processing with non-identical job sizes using simulated annealing. *International Journal of Production Economics*, 87, 141–147.
- Mokhtari, H., & Salmasnia, A. (2015). A Monte Carlo simulation based chaotic differential evolution algorithm for scheduling a stochastic parallel processor system. *Expert Systems with Applications*, 42(20), 7132–7147.
- Mönch, L., Balasubramanian, H., Fowler, J. W., & Pfund, M. E. (2005). Heuristic scheduling of jobs on parallel batch machines with incompatible job families and unequal ready times. *Computers & Operations Research*, 32, 2731–2750.
- Mönch, L., Unbehaun, R., & Choung, Y. I. (2006). Minimizing earliness-tardiness on a single burn-in oven with a common due date and maximum allowable tardiness constraint. *OR Spectrum*, 28, 177–198.
- Santos Coelho, L. D. (2008). A quantum particle swarm optimizer with chaotic mutation operator. *Chaos, Solitons and Fractals*, 37, 1409–1418.
- Tang, M., Xin, Y., Li, J., & Zhai, J. (2013). Nonconvex resource control and lifetime optimization in wireless video sensor networks based on chaotic particle swarm optimization. *Applied Soft Computing*, 13, 3273–3284.
- Tasgetiren, M. F., Liang, Y. C., Sevkli, M., & Gencyilmaz, G. (2007). A particle swarm optimization algorithm for makespan and total flowtime minimization in the permutation flowshop sequencing problem. *European Journal of Operational Research*, 177, 1930–1947.
- Tavakkoli-Moghaddam, R., Rahimi-Vahed, A., & Mirzaei, A. H. (2007). A hybrid multi-objective immune algorithm for a flow shop scheduling problem with bi-objectives: Weighted mean completion time and weighted mean tardiness. *Information Sciences*, 177, 5072–5090.
- Turgut, O. E., Turgut, M. S., & Coban, M. T. (2014). Chaotic quantum behaved particle swarm optimization algorithm for solving nonlinear system of equations. *Computers and Mathematics with Applications*, 68, 508–530.
- Van den Bergh, F. (2006). *An analysis of Particle Swarm optimizers*. Thesis (Ph.D.), University of Pretoria.
- Wang, C. S., & Uzsoy, R. (2002). A genetic algorithm to minimize maximum lateness on a batch processing machine. *Computers & Operations Research*, 29, 1621–1640.
- Wang, M. H., & Chou, D. F. (2010). Solving the parallel batch-processing machines with different release times, job sizes, and capacity limits by metaheuristics. *Expert Systems with Applications*, 37, 1510–1521.
- Wang, W.-X., Wang, X., Ge, X.-L., & Deng, L. (2014). Multi-objective optimization model for multi-project scheduling on critical chain. *Advances in Engineering Software*, 68, 33–39.
- Wang, Y., Zhou, J., Qin, H., & Youlin, L. (2010). Improved chaotic particle swarm optimization algorithm for dynamic economic dispatch problem with valve-point effects. *Energy Conversion and Management*, 51, 2893–2900.
- Xiang, T., Liao, X. F., & Wong, K. W. (2007). An improved particle swarm optimization algorithm combined with piecewise linear chaotic map. *Applied Mathematics and Computation*, 190, 1637–1645.
- Xu, S. and Bean, J. C., (2007). A genetic algorithm for scheduling parallel non-identical batch processing machines. In *Proceedings of the 2007 IEEE symposium on computational intelligence in scheduling (CI-Sched 2007)* (pp. 143–150).
- Yuan, X., Yuan, Y., & Zhang, Y. (2002). A hybrid chaotic genetic algorithm for short-term hydro system scheduling. *Mathematics and Computers in Simulation*, 59, 319–327.